



Guia de Revisão: Programação para Engenharia (P1) Atualizada

1. Primeiros Comandos e Fundamentos

Tipos de Variáveis Primitivas

- `str()` : Strings (textos). Ex: `'Olá'` , `'7.5'` , `"Python"` .
- `int()` : Números inteiros. Ex: `7` , `-30` , `12232` .
- `float()` : Números de ponto flutuante (decimais). Ex: `4.5` , `0.0045` , `7.0` .
- `bool()` : Valores booleanos. Apenas `True` ou `False` .

Entrada de Dados (`input`)

- `A = input('Digite o nome')` : Recebe sempre uma **string**.
- `Idade = int(input('Digite a idade'))` : É necessário converter para o tipo numérico desejado para realizar cálculos.

Saída de Dados (`print`)

O comando `print()` exibe mensagens e variáveis no console.

Uso Básico: * `print('Olá Mundo')` : Exibe uma string. * `print(variavel)` : Exibe o valor de uma variável. * `print(valor1, valor2, valor3)` : Exibe múltiplos valores separados por espaço.

F-strings (Formatação com Variáveis): Introduzidas no Python 3.6, as f-strings permitem inserir expressões diretamente em strings usando chaves `{}` .

```
nome = 'Maria'
idade = 25
print(f'Meu nome é {nome} e tenho {idade} anos') # Meu nome é Maria e t

x = 3.14159
print(f'Pi com 2 casas: {x:.2f}') # Pi com 2 casas: 3.14
```

Formatação de Números: * `:.2f` → Decimal com 2 casas (float). * `:.0f` → Sem

casas decimais.

Outros Métodos de Formatação:

Método `.format()`:

```
print('Olá {}, você tem {} anos'.format('João', 30)) # Olá João, você tem 30 anos
print('O valor é {:.2f}'.format(3.14159)) # O valor é 3.14
```

Método com `%` (mais antigo):

```
print('Olá %s, você tem %d anos' % ('João', 30)) # Olá João, você tem 30 anos
print('Valor: %.2f' % (3.14159,)) # Valor: 3.14
```

Quebra de Linhas e Caracteres Especiais: * `\n` → Nova linha. * `\t` → Tabulação (indentação). * `\\` → Barra invertida literal. * `\'` / `\"` → Aspas literais.

```
print('Linha 1\nLinha 2') # Exibe em duas linhas
print('Nome\tIdade\nMaria\t25') # Exibe tabulado
```

Operadores Aritméticos

1. `+` : Adição
2. `-` : Subtração
3. `*` : Multiplicação
4. `/` : Divisão
5. `**` : Potenciação
6. `//` : Divisão Inteira (descarta as casas decimais)
7. `%` : Resto da Divisão (Módulo)

Ordem de Precedência

1. `()` : Parênteses
 2. `**` : Potência
 3. `*` , `/` , `//` , `%` : Multiplicação e divisões
 4. `+` , `-` : Adição e subtração
- ////////////////////////////////////

2. Loops e Condicionais

Condicional `if`

Estrutura para tomada de decisões baseada em valores booleanos.

```
if (condicao):
    # Execução se for True
elif (outra_condicao):
    # Execução se a primeira for False e esta True
else:
    # Execução caso todas as anteriores sejam False
```

Laços de Repetição

for: Utilizado quando se conhece o intervalo ou a sequência.

```
range(inicio, fim, passo)
```

```
range(1, 10) -> 1 até 9 (o fim é exclusivo).
```

```
range(10, 1, -1) -> Contagem regressiva.
```

while: Repete o bloco enquanto uma condição for verdadeira.

3. Variáveis Compostas

Tuplas `()` : Imutáveis (não podem ser alteradas após a criação).

Listas `[]` : Mutáveis.

- `lista.append(x)` : Adiciona ao final.
- `lista.insert(pos, x)` : Insere em posição específica.
- `lista.pop(i)` : Remove pelo índice.
- `lista.remove(x)` : Remove pelo valor.
- `copia = lista[:]` : Forma correta de copiar uma lista sem criar apenas uma referência.

Dicionários `{}` : Estrutura Chave-Valor.

- `contato.keys()` : Retorna as chaves.
- `contato.values()` : Retorna os valores.
- `contato.items()` : Retorna os pares (chave, valor).

4. Funções

Definição de blocos de código reutilizáveis:

- Definição: `def nome_da_funcao(args):` .
 - Retorno: `return resultado` .
 - Argumentos Variáveis: `def func(*a)` recebe múltiplos valores como uma tupla.
 - Recursividade: Função que chama a si mesma (ex: cálculo de Fatorial).
 - Lambda: Funções anônimas simplificadas. Ex: `f = lambda a, b: (a + b)` .
- ////////////////////////////////////

5. NumPy (Computação Científica)

Biblioteca voltada para processamento de matrizes e arrays multidimensionais.

Criação de Arrays

- `np.array([list])` : Cria array a partir de lista.
- `np.arange(start, stop, step)` : Sequência numérica.
- `np.linspace(start, stop, num)` : num elementos linearmente espaçados.
- `np.random.randint(min, max, shape)` : Inteiros aleatórios.

Operações e Manipulação

- Vetorização: Operações aritméticas aplicadas elemento a elemento automaticamente.
 - Multiplicação de Matrizes: `np.dot(A, B)` .
 - Estatística: `.min()` , `.max()` , `.sum()` , `.mean()` (média), `.std()` (desvio padrão).
 - Reshape: `.reshape(linhas, colunas)` para alterar a forma dos dados.
 - Concatenação: `np.vstack()` (vertical) e `np.hstack()` (horizontal).
- ////////////////////////////////////

6. Matplotlib (Visualização de Dados)

Módulo principal: `import matplotlib.pyplot as plt` .

Tipos de Gráficos Principais

- `plt.plot(x, y)` : Gráfico de linha.
- `plt.scatter(x, y)` : Gráfico de dispersão (pontos).
- `plt.bar(x, y)` / `plt.barh(x, y)` : Barras verticais ou horizontais.
- `plt.pie(x)` : Gráfico de pizza (proporções).
- `plt.hist(dados)` : Histograma para análise de distribuição.

Customização e Formatação

- `plt.title('Texto')` : Título superior.
- `plt.xlabel('Eixo X')` / `plt.ylabel('Eixo Y')` : Legendas dos eixos.
- `plt.grid()` : Habilita a grade.
- `plt.legend()` : Exibe a legenda (necessário definir label no plot).
- `plt.xlim(min, max)` / `plt.ylim(min, max)` : Define limites de visualização.
- `plt.subplot(lin, col, pos)` : Organiza múltiplos gráficos na mesma janela.
- `plt.style.use('nome_do_estilo')` : Muda o tema visual (ex: 'ggplot', 'bmh').